マイコンシステム
第１３回

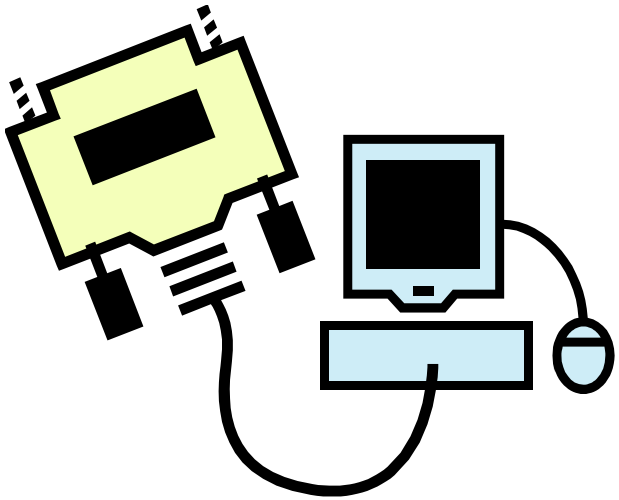
青森大学　ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

目次

- 講義
  - 実装①
  - 状態遷移の実装
- 例題
  - 各状態の処理内容
- 課題
  - 状態遷移の動作確認①



講義

実装①

状態とイベントに応じた

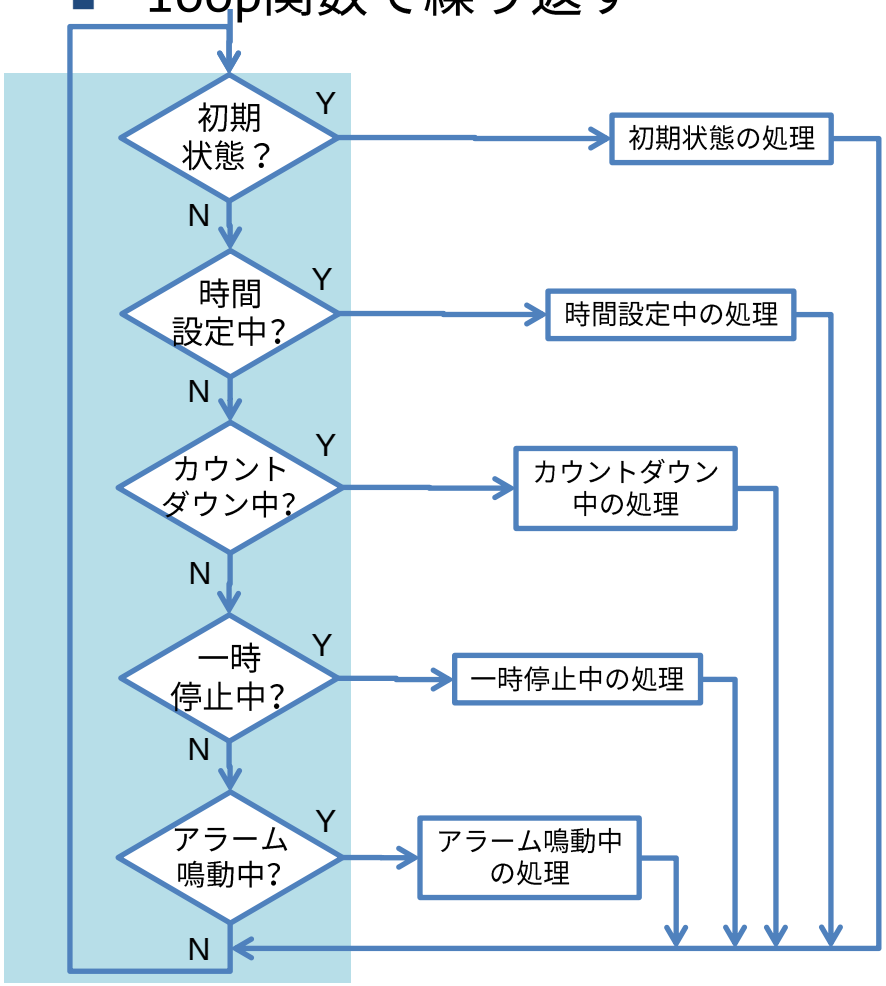
処理を実装する

組込みシステム開発

週	テーマ	内容
7	キッチンタイマーの組立	キッチンタイマーのハードを製作・確認
8 9	要求分析・外部設計	キッチンタイマーに必要な機能を抽出 画面表示、ボタンを押した時の動作を設定 機能の実現手順をフローチャートで図示
10 11 12	内部設計	状態遷移図・状態遷移表の使い方 システムの内部構造を考える 機能の実現手順をより詳細に決める
13 14	実装	プログラムを作る
15	テスト・完成	設計通りにできているか動作検証する

C言語での状態遷移の実装

- 状態遷移を分岐で表す
- loop関数で繰り返す




```
switch (状態変数) {
   case STATE_INIT:  //初期状態
   //初期状態の処理をここに書く
     break;

   case STATE_SETTING: //時間設定中
   //時間設定中の処理をここに書く
     break;

   case STATE_COUNTDOWN: //カウントダウン中
   //カウントダウン中の処理をここに書く
     break;

   case STATE_PAUSE: //一時停止中
   //一時停止中の処理をここに書く
     break;

   case STATE_ALARM: //アラーム鳴動中
   //アラーム鳴動中の処理をここに書く
     break;

   default:  //それ以外
   //それ以外の処理をここに書く
     break;
 }
```

各状態の処理内容

1. 今の状態で必ず実行する処理
2. イベント発生時の処理
   - ① イベントが発生した場合(ｉｆ文を使った分岐)
   - ② 指定のアクションを実行
   - ③ 今の状態を出るとき1回だけ実行する処理（退状処理）
   - ④ 次の状態へ移る（状態遷移）
   - ⑤ 次の状態に入るとき1回だけ実行する処理（入状処理）
   - ⑥ 別のイベントが発生した場合はさらに①～⑤を繰返す

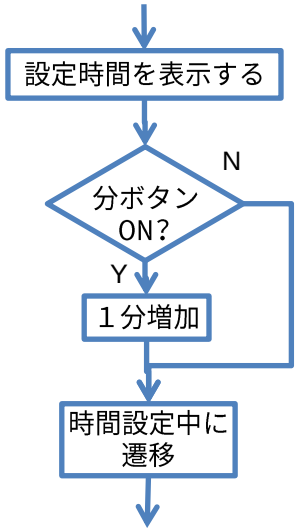
初期状態・時間設定中・カウントダウン中・一時停止中・アラーム鳴動中の
各状態の処理内容を状態遷移図・表・フローチャートで確認する

例題

各状態での処理

初期状態の処理内容

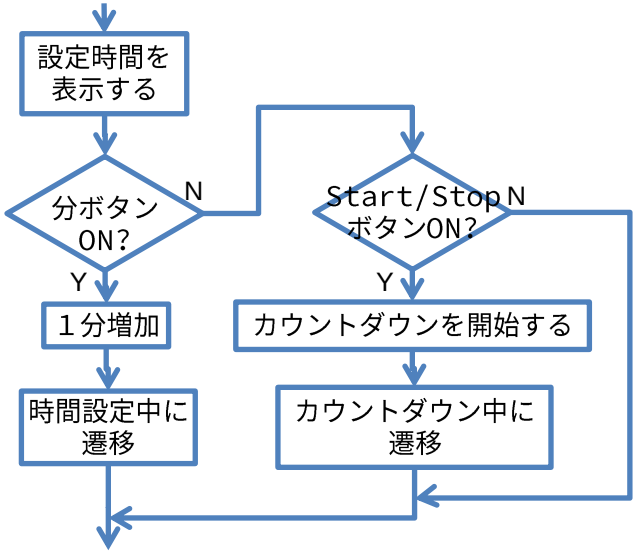
1. 必ず実行する処理
   - 設定時間を表示する
2. イベント発生時の処理
   - 分ボタンを押したら1分増加する
   - 時間設定中に遷移
   - それ以外に状態を出るとき・入るときに1回のみ実行する処理を書ける




```
//初期状態の処理
  case STATE_INIT :
  // 今の状態で必ず または継続して実行する処理
  displayTimer(time, s);

  //イベント発生時の処理
  //分ボタンを押した場合
  if (e == EVENT_PUSH_MINUTE_BUTTON) {
    //1分増加する
    addOneMinute();

    //今の状態を出るときに1回だけ実行する処理
      //なし

    //状態遷移
    ret = STATE_SETTING; //時間設定中に遷移

    //次の状態に入るとき1回だけ実行する処理
      //なし
  }
  break;
```

時間設定中の処理内容①

1. 必ず実行する処理
   - 設定時間を表示する
2. イベント発生時の処理
   - 分ボタンを押したら
     1分増加する、時間設定中に遷移
   - Start/Stopボタンを押したら
     カウントダウンを開始する、
     カウントダウン中に遷移

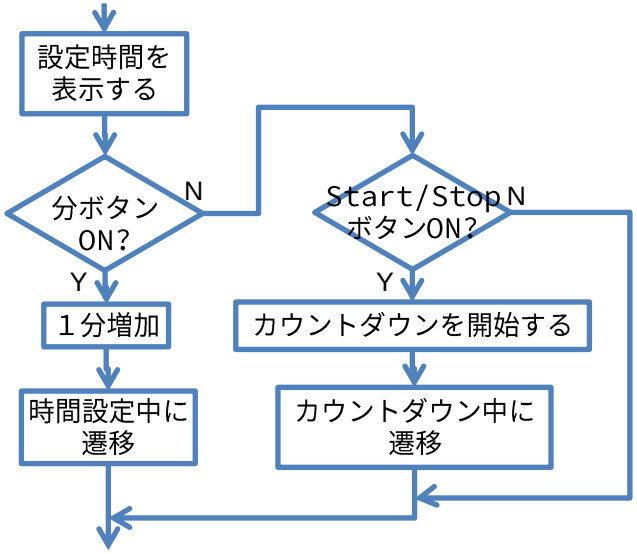



```
//時間設定中の処理
  case STATE_SETTING:
  // 今の状態で必ず または継続して実行する処理
  displayTimer(time, s);

  //イベント発生時の処理
  //分ボタンを押した場合
  if (e == EVENT_PUSH_MINUTE_BUTTON) {
    //1分増加する
    addOneMinute();

    //今の状態を出るときに1回だけ実行する処理
      //なし

    //状態遷移
    ret = STATE_SETTING; //時間設定中に遷移

    //次の状態に入るとき1回だけ実行する処理
      //なし

  //つづく
```

時間設定中の処理内容②

2. イベント発生時の処理

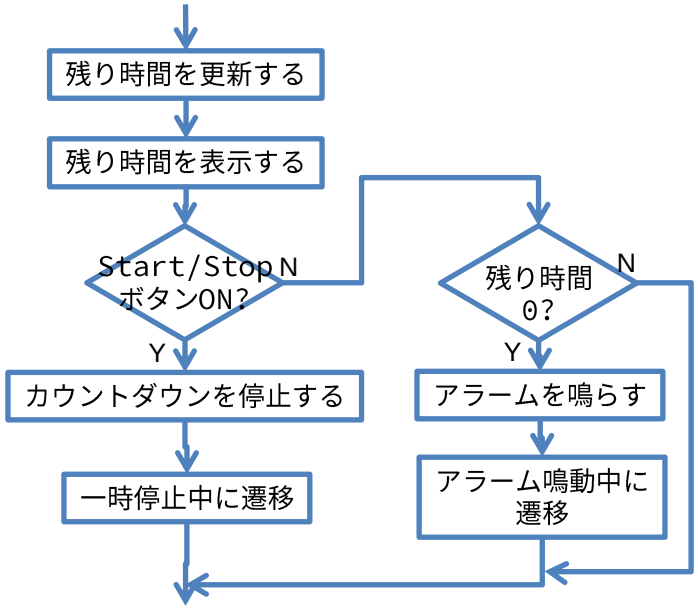
- それ以外に状態を出るとき・入るときに1回のみ実行する処理を書ける




```
//時間設定中の処理（つづき）

  //Start/Stopボタンを押した場合
  } else if (e == EVENT_PUSH_START_STOP_BUTTON) {
    //カウントダウンを開始する
    startTimer();

    //今の状態を出るときに1回だけ実行する処理
      //なし

    //状態遷移
    ret = STATE_COUNTDOWN; //カウントダウン中に遷移

    //次の状態に入るとき1回だけ実行する処理
      //なし
  }
  break;
```

カウントダウン中の処理内容①

1. 必ず実行する処理
   - <u>残り時間を更新する(新規追加)</u>
   - <u>残り時間</u>を表示する
2. イベント発生時の処理
   - Start/Stopボタンを押したらカウントダウンを停止する、一時停止中に遷移

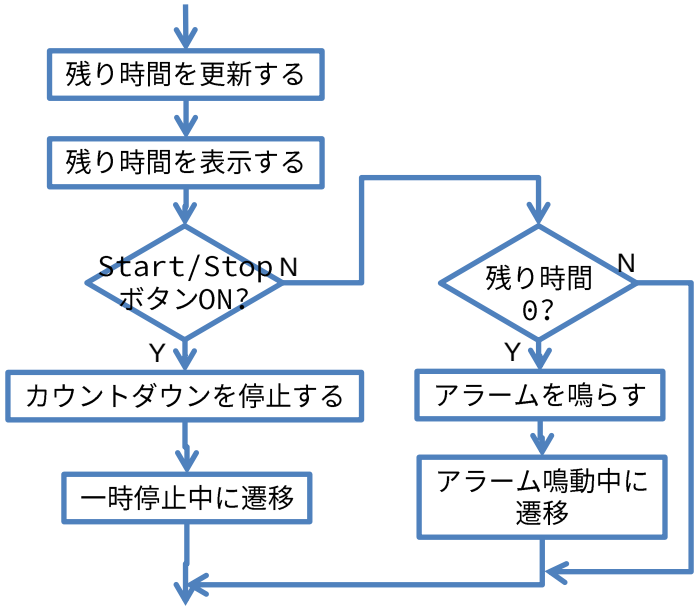



```
//カウントダウン中の処理
  case STATE_COUNTDOWN:
  // 今の状態で必ず または継続して実行する処理
  updateTimer();
  displayTimer(remainTime, s);

  //イベント発生時の処理
  //残り時間が0になった場合
  if (e == EVENT_PUSH_START_STOP_BUTTON) {
    //カウントダウンを停止する
    stopTimer();

    //今の状態を出るときに1回だけ実行する処理
      //なし

    //状態遷移
    ret = STATE_PAUSE; //一時停止中に遷移

    //次の状態に入るとき1回だけ実行する処理
      //なし

  //つづく
```

カウントダウン中の処理内容②

2. イベント発生時の処理

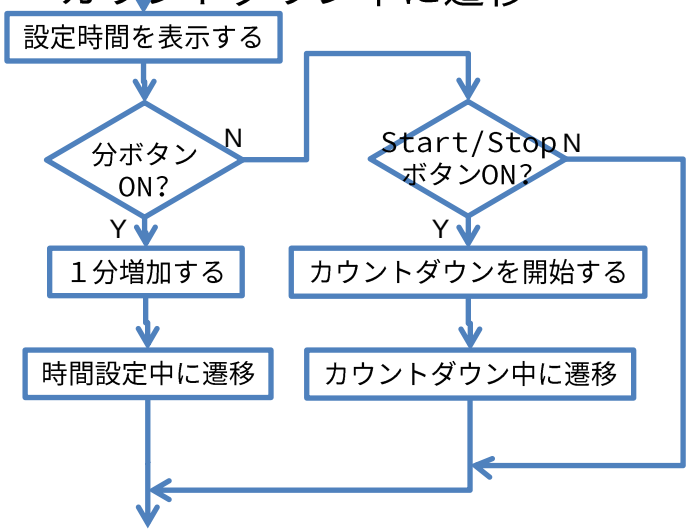
- 残り時間が0になったら
  アラームを鳴らす、
  アラーム鳴動中に遷移
- それ以外に状態を出るとき・入るときに1回のみ実行する処理を書ける




```
//カウントダウン中の処理（つづき）

  //イベント発生時の処理
  //残り時間が0になった場合
  } else if (e == EVENT_TIME_IS_UP) {
    //アラームを鳴らす
    startAlarm();

    //今の状態を出るときに1回だけ実行する処理
      //なし

    //状態遷移
    ret = STATE_ALARM; //アラーム鳴動中に遷移

    //次の状態に入るとき1回だけ実行する処理
      //なし
  }
  break;
```

一時停止中の処理内容①

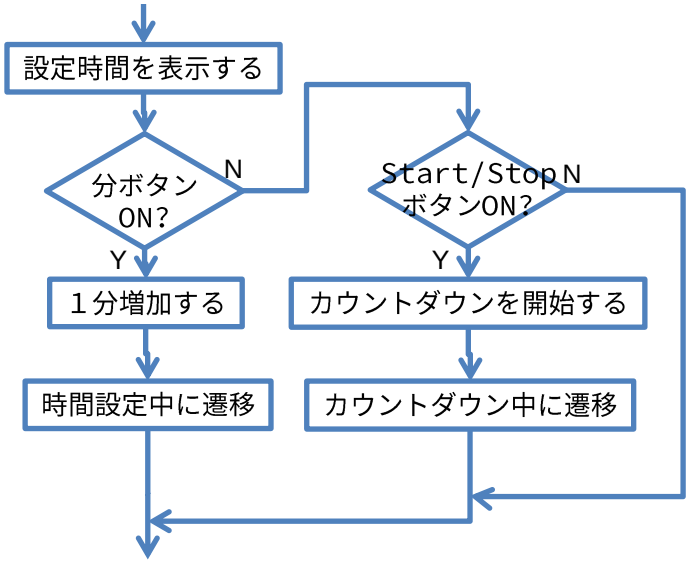
1. 必ず実行する処理
   - 設定時間を表示する
2. イベント発生時の処理
   - 分ボタンを押したら
     1分増加する、時間設定中に遷移
   - Start/Stopボタンを押したら
     カウントダウンを開始する、
     カウントダウン中に遷移




```
//一時停止中の処理
  case STATE_PAUSE:
  // 今の状態で必ず または継続して実行する処理
  displayTimer(time, s);

  //イベント発生時の処理
  //分ボタンを押した場合
  if (e == EVENT_PUSH_MINUTE_BUTTON) {
    //1分増加する
    addOneMinute();

    //今の状態を出るときに1回だけ実行する処理
      //なし

    //状態遷移
    ret = STATE_SETTING; //時間設定中に遷移

    //次の状態に入るとき1回だけ実行する処理
      //なし

  //つづく
```

一時停止中の処理内容②

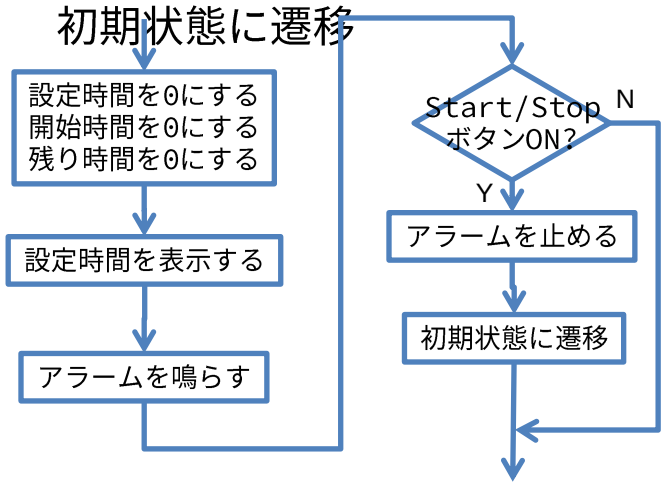
2. イベント発生時の処理
   - それ以外に状態を出るとき・入るときに1回のみ実行する処理を書ける




```
//一時停止中の処理（つづき）

  //イベント発生時の処理
  //Start/Stopボタンを押した場合
  } else if (e == EVENT_PUSH_START_STOP_BUTTON) {
    //カウントダウンを開始する
    startTimer();

    //今の状態を出るときに1回だけ実行する処理
      //なし

    //状態遷移
    ret = STATE_COUNTDOWN; //カウントダウン中に遷移

    //次の状態に入るとき1回だけ実行する処理
      //なし

  }
  break;
```

アラーム鳴動中の処理内容

1. 必ず実行する処理
   - **設定時間、開始時間、残り時間を0にする**
   - 設定時間を表示する
   - アラームを鳴らす
2. イベント発生時の処理
   - Start/Stopボタンを押したら
     アラームを止める、
     初期状態に遷移




```
//アラーム鳴動中の処理
  case STATE_ALARM:
  // 今の状態で必ず または継続して実行する処理
  time = 0L;
  startTimer = 0L;
  remainTime = 0L;
  displayTimer(time, s);
  startAlarm();

  //イベント発生時の処理
  //Start/Stopボタンを押した場合
  if (e == EVENT_PUSH_START_STOP_BUTTON) {
    //アラームを止める
    stopAlarm();

    //今の状態を出るときに1回だけ実行する処理
      //なし

    //状態遷移
    ret = STATE_INIT; //初期状態に遷移

    //次の状態に入るとき1回だけ実行する処理
      //なし
  }
  break;
```

例題のまとめ　メインタブ

```
//メインタブ KT2017_03
enum State {
  STATE_INIT,
  STATE_SETTING,
  STATE_COUNTDOWN,
  STATE_PAUSE,
  STATE_ALARM
} state;

enum Event {
  EVENT_NONE,
  EVENT_PUSH_MINUTE_BUTTON,
  EVENT_PUSH_START_STOP_BUTTON,
  EVENT_TIME_IS_UP
} event;
```

```
long time;
long startTime;
long remainTime;

void setup() {
  setupDevice();
  time = 0L;
  startTime = 0L;
  remainTime = 0L;
  state = STATE_INIT;
  event = EVENT_NONE;
}

void loop() {
  // ここに課題プログラムを追加
}
```

例題のまとめ　Deviceタブ（完成）

```
//Deviceタブ
#include <LiquidCrystal.h>
LiquidCrystal lcd(5, 6, 7, 8, 9, 10, 11);
const int SW1Pin = 2;
const int SW2Pin = 4;
const int LEDPin = 13;
const int SPKPin = 3;

void setupDevice(){
  pinMode(SW1Pin, INPUT);
  pinMode(SW2Pin, INPUT);
  pinMode(LEDPin, OUTPUT);
  lcd.begin(16, 2);
  lcd.clear();
  Serial.begin(9600);
}
void displayString(String str1, String str2){
  lcd.clear();          //画面消去
  lcd.setCursor(0, 0);  //上段左端に移動
  lcd.print(str1);      //文字列str1を表示
  lcd.setCursor(0, 1);  //下段左端に移動
  lcd.print(str2);      //文字列str2を表示
}
```

```
boolean SWIsPressed(int pin){
  if(digitalRead(pin) == HIGH){
    return true;
  } else {
    return false;
  }
}

void startBuzzer(){    //ブザーを鳴らす
  tone(SPKPin, 880);
}

void stopBuzzer(){   //ブザーを止める
  noTone(SPKPin);
}

void lightOn(){      //LEDを点灯する
  digitalWrite(LEDPin, HIGH);
}

void lightOff(){   //LEDを消灯する
  digitalWrite(LEDPin, LOW);
}
```

修正

例題のまとめ　Actionタブ

```
//Actionタブ
void startTimer(){}
void stopTimer(){}
void updateTimer(){}
```

追加

```
void addOneMinute
  time += 60000L;
}
void startAlarm(){
  startBuzzer();
  lightOn();
}
void stopAlarm(){
  stopBuzzer();
  lightOff();
}
```

追加/修正

```
void displayTimer(long ms, enum State s){
  //一部省略
  //ミリ秒を文字列str1に追加
  str1 += String(msec);

  String str2 = "";
  //状態を文字列str2に追加する
  if(s == STATE_INIT){
    str2 = String("STATE_INIT");
  } else if(s == STATE_SETTING){
    str2 = String("STATE_SETTING");
  } else if(s == STATE_COUNTDOWN){
    str2 = String("STATE_COUNTDOWN");
  } else if(s == STATE_PAUSE){
    str2 = String("STATE_PAUSE");
  } else if(s == STATE_ALARM){
    str2 = String("STATE_ALARM");
  } else {
    str2 = String("UNKNOWN");
  }
  //計測時間を1行目に表示、状態を2行目に表示
  displayString(str1, str2);
}
```

追加/修正

例題のまとめ　Eventタブ（完成）

```
//Eventタブ
boolean MinuteButtonIsPressed(){
  static boolean val_old = false;
  boolean val = SWIsPressed(SW2Pin);
  boolean ret = false;
  if(val_old == true && val == false){
    ret = true;
  } else {
    ret = false;
  }
  val_old = val;
  return ret;
}
boolean StartStopButtonIsPressed(){
  static boolean val_old = false;
  boolean val = SWIsPressed(SW1Pin);
  boolean ret = false;
  if(val_old == true && val == false){
    ret = true;
  } else {
    ret = false;
  }
  val_old = val;
  return ret;
}
```

修正

修正

```
boolean TimeIsUp(){
  if(remainTime <= 0L){
    return true;
  } else {
    return false;
  }
}

enum Event receiveEvent(){
  if(MinuteButtonIsPressed()){
    return EVENT_PUSH_MINUTE_BUTTON;
  } else if(StartStopButtonIsPressed()){
    return EVENT_PUSH_START_STOP_BUTTON;
  } else if(TimeIsUp()){
    return EVENT_TIME_IS_UP;
  } else {
    return EVENT_NONE;
  }
}
```

追加

追加

例題のまとめ　Stateタブ①（完成）

追加

```
//Stateタブ
enum State handleEvent(enum State s, enum Event e){
  enum State ret = s;
  switch(s){
    case STATE_INIT:                          //初期状態の場合
      displayTimer(time, s);                  //設定時間と状態を表示
      if(e == EVENT_PUSH_MINUTE_BUTTON){      //分ボタンを押したら
        addOneMinute();                       //設定時間を1分増加する
        ret = STATE_SETTING;                  //時間設定中に遷移
      }
      break;

    case STATE_SETTING:                       //時間設定中の場合
      displayTimer(time, s);                  //設定時間と状態を表示
      if(e == EVENT_PUSH_MINUTE_BUTTON){      //分ボタンを押したら
        addOneMinute();                       //設定時間を1分増加する
        ret = STATE_SETTING;                  //時間設定中に遷移
      } else if(e == EVENT_PUSH_START_STOP_BUTTON){ //Start/Stopボタンを押したら
        startTimer();                         //カウントダウンを開始する
        ret = STATE_COUNTDOWN;                //カウントダウン中に遷移
      }
      break;
```

例題のまとめ　Stateタブ②（完成）

追加

```
case STATE_COUNTDOWN:                          //カウントダウン中の場合
  updateTimer();                               //残り時間を更新する
  displayTimer(remainTime, s);                 //残り時間と状態を表示する
  if(e == EVENT_PUSH_START_STOP_BUTTON){       //Start/Stopボタンを押したら
    stopTimer();                               //カウントダウンを停止する
    ret = STATE_PAUSE;                         //一時停止中に遷移
  } else if(e == EVENT_TIME_IS_UP){            //残り時間が0になったら
    startAlarm();                              //アラームを鳴らす
    ret = STATE_ALARM;                         //アラーム鳴動中に遷移
  }
  break;

case STATE_PAUSE:                              //一時停止中
  displayTimer(time, s);                       //設定時間と状態を表示する
  if(e == EVENT_PUSH_MINUTE_BUTTON){           //分ボタンを押したら
    addOneMinute();                            //設定時間を1分増加する
    ret = STATE_SETTING;                       //時間設定中に遷移
  } else if(e == EVENT_PUSH_START_STOP_BUTTON){ //Start/Stopボタンを押したら
    startTimer();                              //カウントダウンを開始する
    ret = STATE_COUNTDOWN;                     //カウントダウン中に遷移
  }
  break;
```

例題のまとめ　Stateタブ③（完成）

追加

```
  case STATE_ALARM:                          //アラーム鳴動中
    time = 0L;                               //設定時間を0にする
    startTime = 0L;                          //開始時間を0にする
    remainTime = 0L;                         //残り時間を0にする
    displayTimer(time, s);                   //設定時間と状態を表示
    startAlarm();                            //アラームを鳴らす
    if(e == EVENT_PUSH_START_STOP_BUTTON){//Start/Stopボタンを押したら
      stopAlarm();                           //アラームを止める
      ret = STATE_INIT;                      //初期状態に遷移
    }
    break;

  default:                                   //それ以外
    ret = s;                                 //現在の状態をそのまま返す（遷移しない）
    break;
 }
 return ret;                                 //次の状態を返す
}
```

課題

状態遷移の動作確認①

メインタブに追加

- 発生しているイベントを返す関数 receiveEvent()の戻り値を変数eventに代入
- 状態遷移を実装した関数 handleEvent(state, event)で処理を実行、次の状態を変数stateに代入
- 次の状態に遷移後、繰り返す

```
void loop() {
  //現在発生しているイベントを代入する
  event = receiveEvent();
  //現在の状態stateとイベントeventで処理を実行し、次に遷移する状態を代入する
  state = handleEvent(state, event);
}
```

追加

動作確認

- 状態遷移表の一部（初期状態、時間設定中）を確認する

	初期状態	時間設定中	カウントダウン中	一時停止中	アラーム鳴動中
分ボタンを押す	計測時間を1分増加	計測時間を1分増加	／（無視）	計測時間を1分増加	／（無視）
	時間設定中	時間設定中		時間設定中	
Start/Stopボタンを押す	／（無視）	計測時間を減少する	計測時間を維持する	計測時間を減少する	アラームを止める
		カウントダウン中	一時停止中	カウントダウン中	初期状態
残り時間が0になる	／（無視）	×（不可）	アラームを鳴らす	×（不可）	／（無視）
			アラーム鳴動中		

動作確認　初期状態

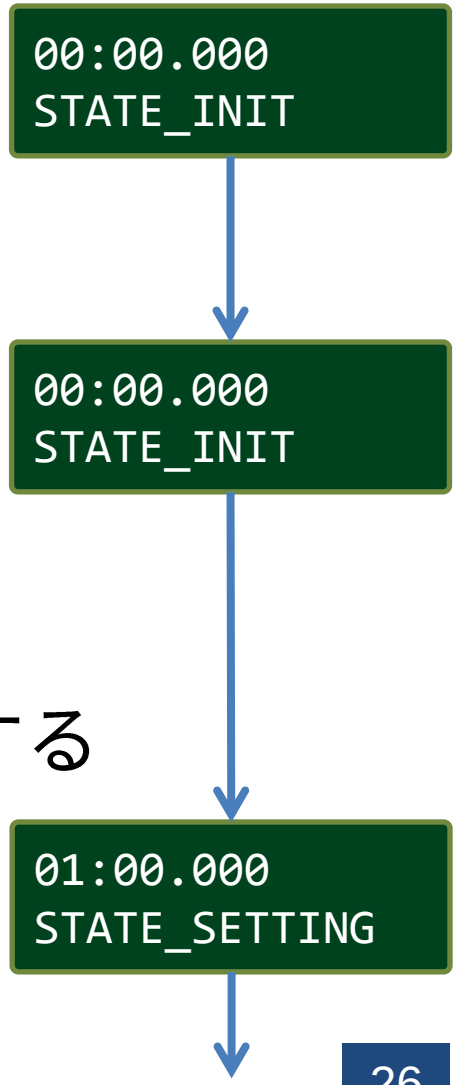
- 初期状態

① Start/Stopボタンを押す

- 時間・状態の表示は変わらない

② 分ボタンを押す

- 時間が1分増加する、複数回確認する
- 状態が時間設定中に遷移する



動作確認　時間設定中

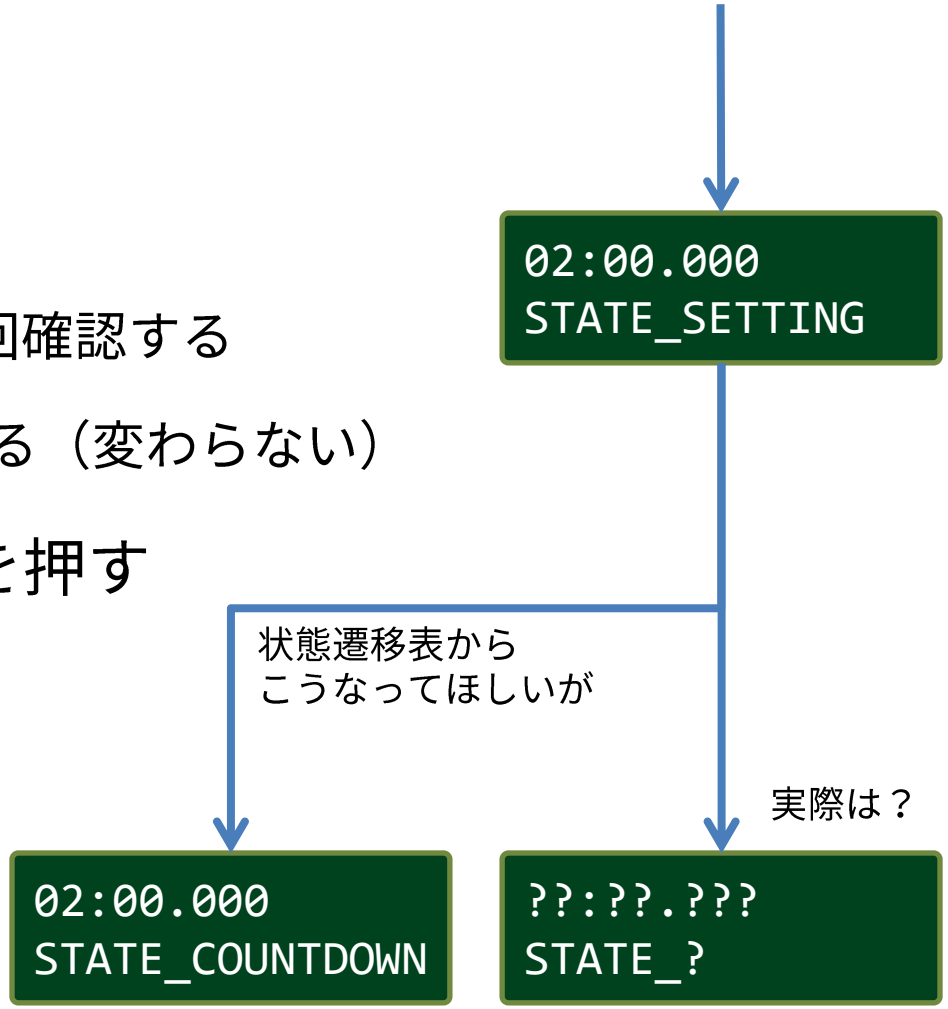
- 時間設定中

① 分ボタンを押す

- 時間が1分増加する、複数回確認する
- 状態が時間設定中に遷移する（変わらない）

② Start/Stopボタンを押す

- <u>時間はどうなるか</u>
- <u>状態はどうなるか</u>



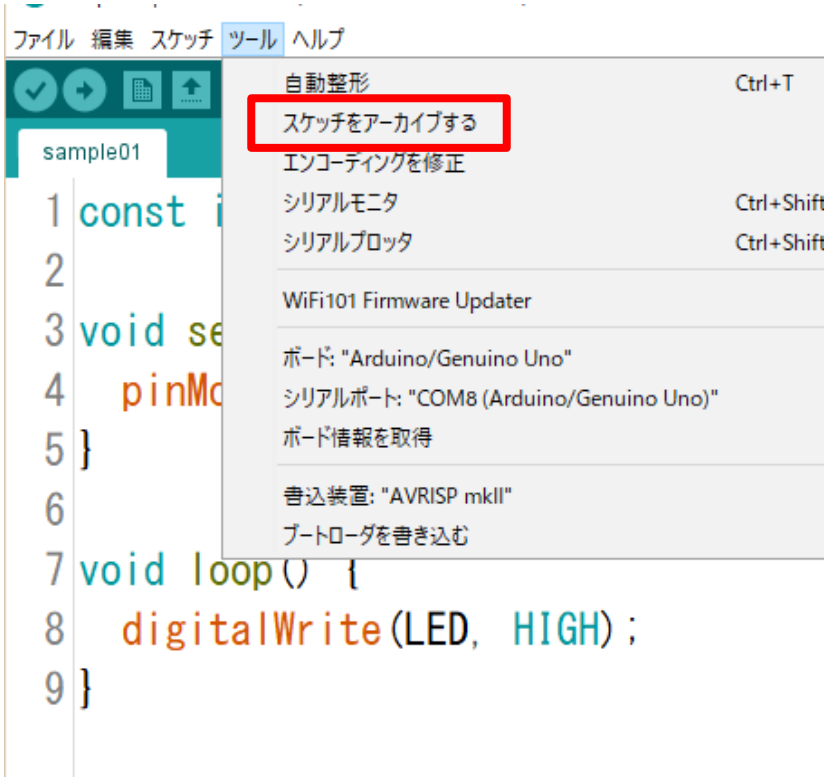
課題の提出方法

時間設定中から遷移した後

時間と状態はどう表示されるか？

プログラムをアーカイブする

- ツール　->
  スケッチをアーカイブ



- 圧縮ファイル名を指定

課題　提出先・期限

- 圧縮ファイルをメールに添付して提出する
  - 課題プログラム KT2017_03
  - 提出前に動作確認してから提出すること
- 提出先
  - haship@aomori-u.ac.jp
  - 件名：マイコンシステム13週
  - 本文に学籍番号、氏名を書く
- 提出期限
  - 次回授業前まで
  - 1/22 13:00まで

コメントの書き方

- 時間と状態がどのように変わったか
- 例
  - 時間
    - 変化前 `02:00.000`　　変化後 `??:??.???`
  - 状態
    - 変化前 `STATE_SETTING`　　変化後 `STATE_???`
- 変化前の時間と状態・変化後の時間と状態をコメントする